

**LAPORAN PRAKTIKUM
TUGAS 5
INTEGRASI DAN DEPLOYMENT WEB SERVICE
STUDI KASUS APLIKASI WISATAKU**



Disusun oleh:
[NAMA MAHASISWA]
NIM: [NIM]

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TINGGI MANAJEMEN INFORMATIKA DAN KOMPUTER
STMIK LOMBOK
PRAYA
2026**

1. Ruang Lingkup

Tugas 5 mencakup Bab 6 (microservices dan API Gateway) dan Bab 7 (Docker, konfigurasi environment, versioning, dan evaluasi siklus hidup API). Status setiap bagian:

Bagian	Status	Bukti
Microservices service-destinasi, service-reservasi (TCP)	Terverifikasi	e2e gateway, smoke test 37/37
API Gateway (ClientProxy, agregasi, penerusan error)	Terverifikasi	e2e gateway, smoke test
Konfigurasi via @nestjs/config	Terverifikasi sebagian	seluruh aplikasi berjalan dengan nilai dari .env; override port lewat environment dipakai e2e dan pemeriksaan gateway di port 3100; penolakan JWT_SECRET kosong diuji unit test. Pesan requireEnv untuk variabel kosong belum diuji terpisah
URI versioning /v1 dan /v2	Terverifikasi	e2e, unit test mapper, request manual
Dockerfile dan docker- compose.yml	Disiapkan, sintaks YAML valid	runtime BLOCKED : Docker belum terpasang
Deployment cloud / URL API aktif	NOT STARTED	butuh VPS
Evaluasi: npm audit	Dijalankan	evaluasi/npm-audit-*.txt
Evaluasi: load test	BLOCKED	k6 belum terpasang

2. Microservices

2.1 Pembagian layanan

Pemecahan mengikuti batas domain bisnis (Tugas 1):

Layanan	Port	Domain	Message pattern
service-destinasi	TCP 5001	Destinasi, Kategori, Ulasan, Fasilitas	destinasi.findAll, destinasi.findOne, destinasi.create, destinasi.update, destinasi.remove, ulasan.findByDestinasi, ulasan.create, fasilitas.findByDestinasi
service-reservasi	TCP 5002	Reservasi	reservasi.create

Modul Bab 6 hanya mencontohkan `destinasi.findAll` dan `destinasi.findOne`. Pola untuk `create/update/remove`, ulasan, dan fasilitas ditambahkan agar seluruh fitur Tugas 3–4 tetap berjalan lewat gateway.

2.2 Logika domain tidak berubah

Bab 6 langkah 2 menyatakan bahwa `DestinasiService` "tetap sama persis — hanya lapisan Controller yang berubah". Proyek ini menerapkannya secara harfiah: service berada di `libs/domain` dan dipakai oleh monolit maupun microservice. Controller microservice hanya memetakan pesan ke service:

```
@MessagePattern(DESTINASI_PATTERNS.findOne)
findOne(@Payload() id: number) {
    return this.destinasiService.findOne(id);
}
```

Nama pattern didefinisikan sekali di `libs/common/src/rpc/patterns.ts` dan dipakai oleh pengirim (gateway) maupun penerima (microservice), sehingga salah ketik terdeteksi saat kompilasi.

2.3 Error lintas layanan

Secara bawaan, microservice NestJS mengubah exception yang bukan `RpcException` menjadi "Internal server error". Akibatnya `NotFoundException` dari service akan sampai ke client sebagai 500. Dua komponen memperbaikinya:

1. `HttpToRpcExceptionHandler` (microservice) mengirim { `statusCode`, `message`, `error` } dari exception asli. Error lain disembunyikan sebagai 500 dan dicatat di log layanan.
2. `sendRpc` (gateway) mengubah payload tersebut kembali menjadi `HttpException` dengan status yang sama. Timeout (`RPC_TIMEOUT_MS`) atau

koneksi gagal menjadi 503 "Layanan internal sedang tidak dapat dihubungi".

Hasilnya, GET /destinasi/999999 lewat gateway tetap 404 dan reservasi dengan tanggal lampau tetap 400. Keduanya diuji di test/wisataku-flow.e2e-spec.ts.

3. API Gateway

Gateway (gateway/) adalah satu-satunya titik masuk HTTP. Tanggung jawabnya:

Fungsi	Implementasi
Routing ke microservice	ClientsModule.registerAsync (Transport.TCP) dengan host/port dari environment; dibungkus DestinasiClient dan ReservasiClient
Autentikasi & otorisasi	AuthModule dari libs/auth (JWT, RBAC); gateway membaca tabel User untuk register/login
Agregasi	GET /destinasi/:id/lengkap memanggil tiga pattern secara paralel (Promise.all)
Versioning	enableVersioning({ type: VersioningType.URI })
Dokumentasi	Swagger /api/docs, GraphQL /graphql dengan skema identik dengan monolit

Gateway diberi DATABASE_URL meskipun contoh docker-compose.yml di modul tidak mencantulkannya. Alasannya, diagram Bab 8.2 menempatkan autentikasi JWT di gateway, sedangkan register/login memerlukan tabel user. Gateway hanya mengakses tabel tersebut; data destinasi dan reservasi selalu melalui microservice.

Data tipe Date berubah menjadi string ISO saat melewati TCP (JSON). Gateway mengembalikannya menjadi Date untuk field tanggal ulasan, karena scalar DateTime GraphQL hanya menerima objek Date.

4. Konfigurasi Environment

Seluruh aplikasi memakai ConfigModule.forRoot({ isGlobal: true, validate }). Tidak ada host, port, atau secret yang ditulis langsung di kode.

Variabel	Aplikasi
DATABASE_URL	semua (wajib)
JWT_SECRET (wajib), JWT_EXPIRES_IN	gateway, monolit

Variabel	Aplikasi
GATEWAY_PORT, RPC_TIMEOUT_MS	gateway
SERVICE_DESTINASI_HOST/PORT, SERVICE_RESERVASI_HOST/PORT	gateway dan microservice terkait

Aplikasi berhenti saat start dengan pesan "Environment variable wajib belum diisi: ..." bila variabel wajib kosong. `.env.example` berisi daftar variabel tanpa nilai rahasia; `.env` dikecualikan di `.gitignore` dan `.dockerignore`.

5. Docker

5.1 Dockerfile multi-stage

Satu Dockerfile per layanan (gateway/, service-destinasi/, service-reservasi/) mengikuti pola Bab 7.3.1:

```
FROM node:22-alpine AS builder
RUN apk add --no-cache openssl
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate
RUN npm run build:destinasi

FROM node:22-alpine
RUN apk add --no-cache openssl
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma
COPY --from=builder /app/dist/service-destinasi ./dist
USER node
EXPOSE 5001
CMD ["node", "dist/main"]
```

Perbedaan dari contoh modul dan alasannya:

Perbedaan	Alasan
node:22-alpine, bukan node:20-alpine	Node.js 20 sudah end-of-life (April 2026)

Perbedaan	Alasan
Build context root repository	Kode <code>libs/</code> dipakai bersama; setiap Dockerfile hanya mem-build aplikasinya sendiri (<code>npm run build:<app></code>)
<code>apk add openssl</code>	Dibutuhkan Prisma engine di Alpine
<code>USER node</code>	Proses tidak berjalan sebagai root
Tidak menyalin folder <code>prisma</code> ke image runtime	Prisma Client sudah membawa skema hasil generate; migrasi dijalankan oleh service migrate

Dependency native `bcrypt` memiliki prebuild untuk Alpine (`linux-x64/bcrypt.musl.node`), sehingga image runtime tidak memerlukan compiler.

5.2 docker-compose.yml

```

mariadb (healthcheck)
  ↳ migrate (sekali jalan: prisma migrate deploy + seed)
    ↳ service-destinasi (5001, internal)
    ↳ service-reservasi (5002, internal)
      ↳ gateway (3000 → host)

```

Keputusan	Alasan
Hanya gateway yang mempublikasikan port	Client tidak boleh mengakses microservice langsung (Bab 6). MariaDB juga tidak dipublikasikan; port 3306 di mesin pengembangan pun sudah dipakai layanan MySQL lain
Service migrate dengan <code>depends_on: service_completed_successfully</code>	Tabel harus ada sebelum layanan menerima request; seed aman diulang
mariadb dengan healthcheck <code>healthcheck.sh --connect --innodb_initialized</code>	<code>depends_on</code> biasa hanya menunggu container start, bukan database siap
<code>--lower-case-table-names=1</code>	Dua migrasi berisi nama tabel huruf kecil karena dibuat di MariaDB Windows; opsi ini menyamakan perilaku di Linux tanpa mengubah migrasi yang sudah diterapkan
<code>\${VAR:?pesan}</code> untuk secret	Compose berhenti dengan pesan jelas bila <code>.env</code> belum diisi; tidak ada secret di file compose

5.3 Status verifikasi

Docker configuration has been prepared and validated syntactically. Runtime verification requires Docker Desktop.

Yang sudah diverifikasi: `docker-compose.yml` di-parse sebagai YAML valid, anchor `DATABASE_URL` ter-resolve untuk keempat service yang membutuhkannya, dan hanya gateway yang memiliki ports. Build yang sama (`npm run build:<app>`) berhasil di luar container.

Yang belum diverifikasi: `docker compose build`, `docker compose up`, migrasi di container MariaDB, dan request ke gateway di container. Perintah pengujiannya ada di `deployment-guide.md`.

6. API Versioning

Perubahan yang disimulasikan (Bab 7.3.4): harga tiket dipecah dari satu nilai `hargaTiket` menjadi `hargaDewasa` dan `hargaAnak`.

	v1 (/v1/destinasi, /destinasi)	v2 (/v2/destinasi)
Harga	<code>hargaTiket</code>	<code>hargaDewasa</code> , <code>hargaAnak</code> (boleh null)
Controller	<code>DestinasiV1Controller (version: ['1', VERSION_NEUTRAL])</code>	<code>DestinasiV2Controller (version: '2')</code>
Sub-resource	<code>/ulasan</code> , <code>/fasilitas</code> , <code>/lengkap</code>	tidak diduplikasi karena tidak berubah

Ini perubahan yang benar-benar *breaking*: client v1 yang membaca `hargaTiket` akan rusak bila menerima struktur v2. Karena itu v2 dibuat sebagai versi baru dan v1 tetap berjalan.

Kolom database tidak diganti nama. `hargaTiket` tetap menjadi sumber harga dewasa, dan migrasi `add_harga_anak` hanya menambah kolom `hargaAnak` yang boleh kosong. Dengan begitu v1 dan v2 membaca data yang sama tanpa migrasi data. Path tanpa prefix (`/destinasi`) dipetakan ke v1 agar client yang dibuat sebelum versioning, termasuk skenario e2e Bab 8.4, tetap berjalan.

Kebijakan deprecation dicatat di `CHANGELOG.md`: v1 ditandai deprecated, tanggal penghapusan belum ditetapkan dan akan diumumkan minimal satu semester

sebelumnya setelah API ter-deploy.

7. Evaluasi Siklus Hidup API (Bab 7.3.5)

7.1 Keamanan: npm audit

npm audit dijalankan pada 30 September 2026. Output asli ada di evaluasi/npm-audit-prod.txt dan evaluasi/npm-audit-all.txt.

Paket rentan	Tingkat	Masuk lewat	Perbaikan otomatis
deepmerge-ts < 8.0.0 (GHSA-ggr8-5vv4-36mx, stack exhaustion saat merge objek rekursif)	high	prisma → @prisma/config	npm audit fix --force menurunkan prisma ke 6.12.0 (breaking)
js-yaml 5.0.0–5.4.0 (GHSA-r3ph-w7gj-g6xm, CPU pada merge key kosong)	moderate	@nestjs/swagger	npm audit fix --force menaikkan ke @nestjs/swagger 12 (breaking)

Total 5 temuan (3 high, 2 moderate); sisanya adalah paket perantara pada rantai yang sama.

Penilaian:

- Rantai deepmerge-ts berada di Prisma CLI (@prisma/config), yang dipakai saat generate dan migrasi, bukan jalur request API. Risiko terhadap API yang berjalan rendah. Tindakan: menunggu rilis Prisma 6.x yang memperbarui @prisma/config, bukan menurunkan versi Prisma.
- js-yaml dipakai @nestjs/swagger untuk endpoint /api/docs-yaml. Input yang diproses berasal dari dokumen OpenAPI milik aplikasi sendiri, bukan dari client. Tindakan: rencanakan upgrade ke @nestjs/swagger 12 sebagai pekerjaan terpisah yang diuji dengan npm run openapi:export dan e2e.

Kedua perbaikan otomatis bersifat breaking, sehingga tidak diterapkan tanpa pengujian.

Tinjauan RBAC: pembagian admin (kelola destinasi) dan wisatawan (reservasi, ulasan) masih sesuai kebutuhan bisnis. Belum ada endpoint untuk melihat atau membatalkan reservasi. Bila fitur itu ditambahkan, diperlukan aturan kepemilikan (wisatawan hanya melihat reservasinya sendiri), tidak cukup hanya pengecekan role.

7.2 Performa: load test

BLOCKED. k6 belum terpasang, sehingga belum ada hasil load test, baik untuk monolit maupun gateway. Tidak ada angka performa yang dilaporkan di dokumen ini. Setelah k6 terpasang, jalankan `load-test.js` terhadap monolit dan gateway (lihat `deployment-guide.md`), lalu bandingkan `p(95)` dan `http_req_failed` keduanya untuk melihat biaya tambahan hop TCP di gateway.

7.3 Keberlanjutan: potensi breaking change

Endpoint / elemen	Potensi perubahan	Dampak
GET/POST/PATCH /v1/destinasi, /destinasi	Penghapusan v1 setelah masa deprecation	Client yang masih membaca <code>hargaTiket</code>
/destinasi tanpa prefix	Dialihkan ke v2 di masa depan	Sama seperti di atas; lebih aman dihapus bersama v1
GET /destinasi tanpa limit	Pagination wajib bila data membesar	Client yang mengharapkan seluruh data dalam satu respons
DELETE /destinasi/:id → 200 + pesan	Diubah menjadi 204 tanpa body	Client yang membaca <code>message</code>
Skema GraphQL <code>Destinasi.hargaTiket</code>	Menyusul struktur v2	GraphQL tidak memakai URI versioning; perlu <code>@deprecated</code> pada field lama lalu field baru ditambahkan
Payload JWT { sub, role }	Penambahan multi-role	Semua guard dan client yang membaca <code>role</code>
Message pattern TCP	Perubahan bentuk payload	Gateway dan microservice harus di-deploy bersamaan

8. Deployment

NOT STARTED. Deployment ke VPS dan URL API aktif memerlukan akses server dan Docker, yang belum tersedia. Langkah yang disiapkan ada di `deployment-guide.md`. Tidak ada URL yang dicantumkan sampai deployment benar-benar dilakukan.

9. Kesimpulan

1. Domain Destinasi dan Reservasi berjalan sebagai microservice TCP di belakang API Gateway. Client hanya mengenal port 3000, dan alur lengkap Bab 8.4 lulus e2e test dengan komunikasi TCP sungguhan.
2. Error microservice diteruskan dengan status aslinya; layanan yang tidak merespons menghasilkan 503, bukan request yang menggantung.
3. URI versioning memisahkan struktur harga lama (v1) dan baru (v2) tanpa migrasi data.
4. Dockerfile dan Docker Compose sudah disiapkan dengan urutan start yang benar dan tanpa secret di repository, tetapi belum dijalankan karena Docker belum terpasang.
5. npm audit menemukan 5 kerentanan pada dependency tidak langsung. Perbaikannya breaking, sehingga dicatat dan direncanakan, belum diterapkan.